

Large References, Fast Checks: Oracle-Guided Porting of a Stochastic Learning System

Anonymous authors

Paper under double-blind review

Abstract

Rewriting a program in another programming language is called porting. How do we check that the new version, the *port*, preserves the original behaviour when the original can give different answers to the same question, and no answer is strictly correct? We investigate this problem while porting Metacat, a system that makes analogies in a micro-domain and learns from previous attempts over the course of an episode. The idea is to collect reference answers once, at some expense, then reuse them for fast checks during development. Good-Turing estimates the chance of an unseen reference answer and guides collection of the set used to flag unexpected port solutions. A p50 set contains the most frequent reference outcomes accounting for at least half the sample; their absence flags potentially missing port solutions. Automating these comparisons removes manual inspection of result distributions after each change. For episodes retaining memory across eight runs, Metacat’s own quality score and conceptual preference select the answers to compare. This makes conceptual behaviour testable without prescribing a fixed expected answer. The main result is practical: the workflow helped repair a direction-handling error and exposed a misleading comparison caused by different execution limits. Historical records do not identify every tested build. Later studies provide 969,000 single-run observations and 25,974 episodes. Only three of 19 problems pass independent episodic coverage validation for both answer definitions. For one validated quality set, none of 1,000 reference-validation episodes selects an answer outside the set, compared with 24 of 100 port episodes; the cause remains unresolved. The process and its application contribute novelty alongside the repair history. We do not quantify the speedup over manual inspection, and we do not establish complete implementation fidelity or better learning.

1 Introduction

If **abc** changes to **abd**, what should happen to **ijk**? One interpretation is to replace the last letter with its successor in the alphabet, giving **ijl**. Another is to replace the last letter with **d**, giving **ijd**. This simple example raises a less simple question: what does it mean for an analogy-making program with basic learning capabilities to give a good answer?

Metacat explores questions like this (Marshall, 1999; Marshall & Hofstadter, 1997). It extends Copycat (Mitchell, 1993; Hofstadter & Fluid Analogies Research Group, 1995), a computational model of analogy-making. For this testing problem, we can view Metacat as a *fuzzy classifier* with an unknown set of answer classes and a simple learning loop.¹ It builds and assesses candidate relationships between letters and groups of letters, rather than choosing from a supplied list of target answers. Memory of previous attempts can influence later attempts. Random choices influence which possibilities it explores. Such a program is called *stochastic*: running it again on the same problem can produce a different result. That variation is part of the design.

¹For a non-academic introduction, see Wikipedia, “Fuzzy classification”. This is a functional view of Metacat’s graded assessments; we make no claim that its answer frequencies are fuzzy-set membership values.

Our engineering task was to rewrite Metacat from the Scheme programming language into Python. A rewrite intended to preserve the original program’s behaviour is called a *port*. We call the Scheme program the *reference* and the Python program the *port*. We wanted the port to keep the reference’s ways of interpreting analogies.

So what should a test expect? One particular answer? The same answer on every attempt? Neither would be a satisfactory test of this system. Nor does giving both programs the same random seed solve the problem. A seed starts a repeatable sequence of pseudorandom numbers, but the two programs may generate different sequences or use their numbers in a different order.

We took a different approach. Run the reference many times and record what it does. Keep that record. As the port changes, use it for fast, automated checks with a much smaller sample. Ask two concrete questions: did the port produce an unexpected solution, and did it fail to produce a common reference solution? Good–Turing guides collection of the reference answers against which we flag the unexpected ones. The p50 rule selects common reference answers whose absence we flag. We explain both below. Either finding gives us something specific to investigate.

In software testing, an *oracle* supplies the expected behaviour against which a test is judged. Our oracle is a collected record, not an authority on every possible answer. It can be incomplete. It can even contain evidence of an error in the reference itself. The word therefore needs care: an oracle flag is a reason to investigate, not a verdict that the port is wrong.

The cost is deliberately unequal: a large, reusable reference and fast, repeated checks. Without an automated comparison, a person must inspect the distribution of answers after each change and decide which differences deserve attention. Our checks do that comparison and name the differences, so human attention goes to investigating them. Collect extensively once, then get actionable feedback after each revision. We have not timed the manual alternative or measured the speedup.

Metacat also remembers previous attempts. A *run* is one attempt to solve a problem. In an *episode*, we run the same problem eight times while keeping memory between attempts, then clear memory before the next episode. Testing only runs that start with empty memory would leave this experience-dependent behaviour untested. By testing learning we mean checking that the port implements this behaviour as the reference does. Whether memory yields better answers, faster learning, or transfer to new problems is a different question, and we do not ask it.

There is a second benefit. A test that requires one fixed expected answer cannot capture a system meant to entertain several analogies and revise its preferences through experience. We instead let Metacat’s own assessments select the episode results we compare. That gives an automated check on conceptual coherence at the level of selected answers: does the port choose among interpretations the way the reference does? No person has to judge every analogy.

The most important result is that this testing approach helped improve the port. Named problem-and-answer differences led to discovering an error in the handling of direction, a repair, and focused tests of that repair. Another detected difference turned out to come from unequal execution limits rather than a faulty analogy rule. Those investigations motivated this paper but, unfortunately, their historical build records are incomplete. Therefore, we designed and executed two additional, properly-documented studies.

These two later studies give a more reproducible account of what the checks find and what they miss. The first study records 969,000 individual runs. The second study records 25,974 episodes and chooses answers using Metacat’s own assessments of quality and conceptual preference. Both studies evaluate fixed, recorded program versions.

There is also a contribution in the process itself. Good–Turing estimation, observed-support sets, and frequency-based selection are established tools; the novelty is how they combine into one reusable testing process for a stochastic port, extended to episodic behaviour. Section 5 compares the closest prior work.

We first explain the testing method with examples, then state its costs and statistical limits. The development history comes next, followed by the two recorded studies. Throughout, we distinguish the answers a program produces from the internal reasoning that produces them; our checks concern the former.

2 Building and Using the Reference

2.1 What We Count

Suppose repeated runs produce the answers `ijl`, `ijd`, and `ijk`. The set $\{\text{ijl}, \text{ijd}, \text{ijk}\}$ records which answers appeared, without recording how often each appeared. A probability *distribution* describes the chance of each possible outcome. In probability theory, its *support* is the set of outcomes that have a nonzero chance of occurring. Our three observed answers belong to the reference’s support, but the reference may also be able to produce answers that we have not yet seen. We therefore call our record the *observed support*.

The counts matter too. Seeing `ijl` 900 times is different from seeing it once. We keep both the set and the count for each result. An *outcome* is the thing we choose to record from an attempt. Usually it is an answer string, but not every attempt ends with an answer.

Metacat works through small computational tasks called *codelets*, which build or assess parts of a proposed analogy. We limit how many codelets a run may execute; that limit is the *cap*. The historical data use `*NONE*` when execution stops without an answer and `*CAP*` when it reaches the codelet limit. The later single-run study also uses `*ERROR*` for a failure inside the program. These outcomes stay in the counts. An error additionally raises a separate error flag; we never require the port to reproduce a reference defect.

Here is the notation used below. Let x be one analogy problem and o one recorded outcome. Write $p_R(o \mid x)$ for the probability that the reference produces o , and $p_P(o \mid x)$ for the corresponding port probability. The subscripts distinguish the two programs; the vertical bar means “given this problem.” Both probabilities also depend on the recorded program version, settings, memory, execution limit, and definition of an outcome.

After N_x reference runs, $c_x(o)$ is the number of times outcome o appeared. The observed support set and its estimated frequencies are

$$S_x = \{o : c_x(o) > 0\}, \quad \hat{p}_R(o \mid x) = c_x(o)/N_x.$$

Here S_x is the *observed support set for problem x* : the outcomes seen at least once in its reference sample. The estimate $\hat{p}_R$ is each outcome’s share of that sample. A hat denotes an estimate. We use n_x for the smaller port-check size and J for the number of problems. The true support is $\text{supp}(p) = \{o : p(o) > 0\}$, in a countable space of possible outcomes $\mathcal{O}$.

Recording answer letters loses information. Two answers with the same letters may come from different rules, different internal structures, or different sequences of decisions. This test does not compare all of those details. Even a very large or infinite answer space can be manageable if most answers are rare in total. A space with probability spread thinly across many answers may be much harder to sample adequately.

2.2 How Good–Turing Guides Collection

To flag unexpected port solutions usefully, we first need to explore what the reference can produce; otherwise an unfamiliar port answer may only expose a poorly sampled reference. How much reference sampling is enough? We use Good–Turing estimation (Good, 1953) to help decide. Its simplest missing-mass estimate counts *singletons*: distinct outcomes that have appeared exactly once so far. If five answer types have each appeared once in 100 runs, then $f_1/N = 5/100 = 0.05$. This estimates the chance that the next reference run produces an outcome we have not yet seen. It does not estimate how many possible answer types remain undiscovered.

An answer stops being a singleton when it appears again. If the only once-seen answer reappears on the next run, f_1 falls from one to zero. A zero estimate therefore means only that no observed type has count one.

More formally, $f_{1,x} = |\{o : c_x(o) = 1\}|$ counts the singleton types. The reference’s *missing mass* is the total probability of all outcomes outside our observed set. Its value and the estimate are

$$M_{R,x} = \sum_{o \notin S_x} p_R(o \mid x), \quad \widehat{M}_{R,x} = f_{1,x}/N_x.$$

The distinction matters. Missing mass is a property of the unknown reference distribution; the singleton ratio is a number calculated from one sample. It is useful guidance; it is no upper confidence bound. Our construction rules are *heuristics*: practical decision rules with no guarantee that the desired condition holds when the rule says to stop. The finite-sample analysis of McAllester & Schapire (2000) adds uncertainty terms for independent sampling, and even that does not turn repeated inspection of the raw ratio into a guarantee.

The historical single-run collector used these rules:

1. After at least 3,000 runs, stop if there is a singleton and $f_{1,x}/N_x \leq 10^{-4}$, or 0.0001.
2. If there are no singletons, require at least 10,000 runs before stopping.
3. Record an overshoot below 6×10^{-5} , but do not treat it as a second acceptance condition. Use a 50,000-run checkpoint and a 200,000-run hard maximum to limit spending.

Eight worker processes sampled separate, interleaved seed indices. A coordinator merged their counts and calculated one overall singleton ratio; workers did not make independent stopping decisions. Work already underway could finish after a stopping condition was reached.

The 10,000-run minimum sounds substantial, but it does not certify the target. Suppose there are just two outcomes, with probabilities 0.9998 and 0.0002. The chance of seeing only the first in 10,000 runs is

$$(0.9998)^{10000} \simeq 0.1353.$$

In that case, the estimate is zero, although the actual missing mass is 2×10^{-4} , twice the intended target. Also, one singleton at 10,000 runs satisfies the positive-singleton rule; the minimum is not justified by rejecting that case.

The historical files contain counts and codelet-use summaries but no discovery order, so they cannot support a retrospective comparison of stopping rules. The later single-run study preserves order and permits the comparisons in Section 7.4.

The later episodic study deliberately allowed an earlier stop: check after every episode, with no minimum, and *freeze* the observed set as soon as $f_1/N \leq 10^{-4}$. Two identical first answers can therefore stop collection with a one-answer set. We call the frozen set a *candidate oracle*; whether it covers reference behaviour is a separate question, answered by new reference observations.

2.3 Which Common Answers Should a Check Look For?

Unexpected solutions are only half of the comparison. A port may also lose solutions the reference regularly produces, and the p50 check targets those. A fast check cannot be expected to repeat every rare reference answer, so we select a set of common outcomes and ask whether each appears at least once. The dissertation’s alternative rules and analogy families (Marshall, 1999, Chapters 3 and 5) led us to expect broad answer sets; checking only the common answers keeps that breadth from forcing a new reference collection after every code change. Rank outcomes from most to least frequent, then take the shortest initial list accounting for at least half of the reference observations. We call this the *p50 head*; the “50” is half of the observations. Equal counts are ordered by outcome text, in fixed dictionary order.

For a small worked example, suppose 100 reference observations contain 40 *ijl*, 25 *ijd*, 20 *ijk*, and 15 *ijm*. The observed set contains all four answers. The p50 head contains *ijl* and *ijd*, whose counts add to 65; *ijl* alone accounts for only 40.

Now suppose a port check produces ijl , ijk , and ijn . It missed the selected common answer ijd , and produced ijn , which was outside the reference record. Those two findings are the report: two questions to investigate.

Let H_x be the p50 head and T_x the set of outcomes observed in the port check. The reports are

$$\text{MISSING}_x = H_x \setminus T_x, \quad \text{NOVEL}_x = T_x \setminus S_x.$$

The symbol $\setminus$ means set difference: members of the first set that are absent from the second. **MISSING** means “a selected reference result was not seen in this check.” **NOVEL** means “a check result was not seen in the reference sample”, which is weaker than genuinely new. A check with no flags establishes only these memberships.

We freeze the reference set and head before checking the port. Adding each unexpected answer as soon as we see it would erase the evidence we wanted to study. The report instead retains the problem, answer, count, seeds, settings, reference size, stopping reason, and singleton ratio. Investigation may reveal a reference omission, a reference defect, a port defect, or mismatched test settings.

The p50 rule does not guarantee that each head member is common enough to appear reliably within the check budget: if B answers are equally likely, each has probability $1/B$, head member or not. A boundary answer can also move in or out when the reference is resampled; in the historical data, `copy5/cc` sits exactly on the half-mass boundary.

2.4 What Counts as an Episode’s Answer?

Memory changes the sampling question. The second run in an episode starts with experience from the first, and the third starts with experience from both. Those eight runs are not eight independent observations of one fixed distribution, so we treat the *whole episode* as one observation. Every new episode starts from the specified initial memory M_0 , uses the same problem schedule, and lasts a fixed number L of runs.

We must then decide which feature of the episode to compare. A *projection* is a rule for turning an episode into one recorded result. The historical study selected the last successful answer among eight runs. That is an endpoint within a fixed budget; the system may not have converged on it. It also needs a separate record of episodes that never answer and of runs that end at the cap.

The later study asks a more conceptual question: which answer did Metacat itself judge best? We use two definitions, each based on the program’s own, or *native*, evaluation:

1. **best_a** is the answer with the highest numerical quality score in the episode. We call it the *quality winner*.
2. **best_b** is an answer that no other answer defeats under Metacat’s own pairwise conceptual-preference comparison. We call it the *preference winner*. This need not be the answer with the highest numerical score.

For ties, select the eligible answer reached first. Each complete episode that produces any answer supplies one selected string to each of two frequency tables; the two strings may be identical, and our analysis never assumes they are independent.

Why these definitions? The last answer is easy to record, but being last is no conceptual justification; quality and preference let Metacat’s own assessments justify the choice. Within each table, distinct types are distinct selected letter strings, which still merges different structures with the same letters. That is a deliberate choice about what to test; Section 8 asks whether a better population exists.

These selections make the conceptual comparison repeatable without a human reviewer and without prescribing one answer per episode. We ask whether the port chooses from the same observed answer populations when quality and conceptual preference pick the winners. Fixed expected-answer assertions leave that aspect of conceptual coherence unspecified.

Let $\mathcal{E}_{x,e}$ denote episode e , including its answers and memory changes, and let $Z_{x,e} = \phi(\mathcal{E}_{x,e})$ be its selected result under the chosen projection ϕ . Episode counts and observed sets follow the same pattern as run counts:

$$c_x^\phi(z) = \sum_{e=1}^{N_x} \mathbf{1}\{Z_{x,e} = z\}, \quad S_x^\phi = \{z : c_x^\phi(z) > 0\}.$$

The indicator $\mathbf{1}$ adds one when the selected result equals z and zero otherwise. Here N_x counts episodes rather than the LN_x runs inside them; for quality and preference winners, it counts complete episodes with an answer. Answerless episodes and program errors stay in separate records, so every coverage statement is conditional on completing an episode with an answer.

Resetting between independently randomized episodes makes it reasonable to model them as draws from one episode distribution, provided no hidden state leaks between episodes. This uses established reset-based testing ideas (Gerhold & Stoelinga, 2018), not a new independence theorem. Good–Turing and head selection apply to these episode results, not to all inner runs pooled together.

These checks never compare complete answer sequences; claiming agreement in retrieval, memory updates, or the route through an episode would need richer observations and memory-specific controls. Letting memory grow across episodes would be a different problem, in which both the dependence and the distribution could change. Results for stationary Markov sequences (Pananjady et al., 2024), a model of dependence with an unchanging distribution, do not automatically cover that case.

2.5 Checking the Reference With New Data

A low singleton ratio suggests that collection might stop; it says nothing with stated confidence about whether the frozen set is adequate. For that second question we run the reference again with new randomness after freezing the candidate set. These *held-out validation* observations played no part in building the set they test.

Each validation winner outside the frozen set counts as a miss, every time it occurs. From the number of misses in a fixed-size sample, we calculate a one-sided binomial upper confidence bound on the probability of another outside result; one-sided means the bound is an upper limit only. A set *qualifies* only if that upper bound meets the declared coverage target; failed or absent validation supplies no claim.

The episodic study uses a coverage target of 0.01, or 1%, looser than the 0.0001 construction heuristic. It considers 19 problems and two answer definitions per problem: 38 populations in all. To make simultaneous 95% confidence statements, we allocate a total error allowance of 0.05 equally across those 38 populations. This Bonferroni allocation uses $\alpha = 0.05/38$ for each bound and does not require independence between the two winner definitions. We keep the allocation fixed even for populations that never become eligible for validation.

Under this study’s rule, a problem receives validation only when both answer sets have frozen. One definition can pass validation while the other fails. Passing means that the reference’s conditional chance of an outside winner meets the bound; it does not establish port correctness, validate a p50 head, or transfer the bound to a port with a different answer distribution.

Validation is part of building the reference and belongs in its cost. With zero misses in m independent validation draws, the one-sided $(1 - \alpha)$ upper limit is $1 - \alpha^{1/m}$. For a single population at $\alpha = 0.05$, achieving a limit of 10^{-4} requires 29,956 draws. Asking for simultaneous confidence across several populations requires more evidence. The later single-run study uses 30,000 validation runs per problem; its nominal and simultaneous results appear in Section 7.2.

The process is straightforward to state, although its limits matter:

Build the reference for each problem:

Collect outcomes under a recorded sampling rule and budget.

Freeze the observed set and, where used, its p50 head.
 Record counts, settings, stopping reason, and singleton ratio.
 Use separate reference observations to assess coverage.
 Keep failed or missing validation visible.
 Do not add validation discoveries to erase misses.

Check a port revision:

Use the same outcome definitions and compatible test settings.
 Collect the budgeted sample of runs or whole reset episodes.
 Report outcomes outside the frozen reference set.
 Where a p50 head is used, report its absent members.
 Keep enough evidence to investigate each finding.
 Do not treat a check without flags as proof of equivalence.

3 Why Spend More on the Reference?

The point is iteration speed. Building a reference is expensive, and so is inspecting result distributions by hand after every code change. Once the reference exists, the software counts outcomes, checks membership, and reports missing and unexpected answers; the human’s job shrinks to investigating them.

Test-driven development (TDD) is the closest analogy. In TDD, we write an automated test before implementing the behaviour it asks for, make it pass, then improve the code while keeping the tests passing.² Time spent up front means later checks report failures without a person inspecting every passing result. Our reference makes the same investment for stochastic behaviour. Section 10 considers development beyond porting, and the difference between a sampled reference and a specification.

Automation is the speed advantage over manual review. It does not make a discrepancy statistically significant; that depends on the sampling design and the test. A reference that costs 100 times one check may still pay for itself over many checks, but the arithmetic must include the whole job: collection, validation, storage, running the port, comparing results, and following up the findings.

Let C_R be the one-time reference cost. For check cycle k , let $C_{P,k}$ be the sampling-and-comparison cost and $C_{F,k}$ the cost of investigating its findings. Across K cycles that can legitimately reuse the reference, the total and average costs are

$$C_{\text{total}}(K) = C_R + \sum_{k=1}^K (C_{P,k} + C_{F,k}), \quad \overline{C}(K) = \frac{C_R}{K} + \frac{1}{K} \sum_{k=1}^K (C_{P,k} + C_{F,k}).$$

The first term in the average, C_R/K , spreads the reference cost across checks; that is what *amortized cost* means here. It is an accounting identity. Any fair comparison with another method must let that method reuse its reference too.

In a single-run check the imbalance is N_x reference runs against n_x port runs, usually with $N_x \gg n_x$; for learning-mode tests the units are episodes. Counts are not elapsed time. One episode may cost far more than another, especially when memory changes how long an answer takes, and Scheme versus Python, processor, numerical software, and concurrency all move the cost. We therefore report count ratios only as count ratios.

What can stay fixed while the port changes? We retain reference counts, the observed set, any selected head, program and dependency versions, seeds, problem schedules, initial memory, stopping conditions, and outcome definitions. Together, these specify the question the reference answers. Changing an episode from eight runs to sixteen changes that question, as does changing the initial memory, the cap, or the meaning of an outcome; each needs a new reference or a justified transformation of the old one. A confirmed reference defect invalidates the affected part of the reference.

²See Martin Fowler’s practitioner account, “Test Driven Development”.

There is another limit to reuse. Developers can gradually adapt a port to the particular reference they keep checking. Repeated checks are then not independent new evidence, and their individual statistical claims do not add up to a guarantee over the whole development history. Fresh final audit samples and tests with known defects remain important.

4 What the Flags Can and Cannot Tell Us

A named-answer flag gives us a concrete case to investigate. Interpreting it still requires care about sampling, reference coverage, and the chance of observing the answer within the check budget. This section states those limits.

4.1 When a Common Answer Is Missing

Suppose a frozen head contains an answer with port probability $p_P(o \mid x)$. If the check observations are independent and that probability stays fixed, the chance of missing it in all n_x observations is

$$\Pr(o \notin T_x \mid S_x, H_x) = (1 - p_P(o \mid x))^{n_x}.$$

This is the probability of not seeing the answer once, multiplied across all draws, and it uses the *port's* probability. Substituting the observed reference share $\hat{p}_R(o \mid x)$ gives a comparison number we call a *plug-in baseline*: an estimate plugged into a formula as if it were the true probability. The substitution ignores uncertainty in the reference sample, head selection, and stopping rule, and a reference share is not an established lower bound on the port's chance of an answer, so the baseline is not a calibrated error probability.

If we could justify a lower bound $p_P(o \mid x) \geq \ell_{x,o} > 0$, then $(1 - \ell_{x,o})^{n_x}$ would be an upper bound on absence. Adding these bounds over all selected answers gives a union bound for a check cycle; the absence events themselves need not be independent. Establishing such lower bounds, with their uncertainty, is separate work; support equality alone supplies none.

There is also a difference between a false alarm and a missed defect. Failing to see a still-possible answer can produce a misleading discrepancy flag. If the port has completely lost an answer, so that $p_P(o \mid x) = 0$, then the answer is absent with certainty. Its head-membership test cannot miss that deletion merely because the check sample is small.

4.2 When an Answer Falls Outside the Reference

An outside answer has three useful counts. How many times did it occur? How many different outside answer types appeared? Did the problem receive any outside-answer flag at all? Ten occurrences of one answer are ten draws, one distinct type, and one flagged problem.

For frozen S_x , let $q_{P,x} = \sum_{o \notin S_x} p_P(o \mid x)$ be the port's probability of producing anything outside the reference. With independent check draws, the expected number D_x of outside draws and the probability of at least one are

$$\mathbb{E}[D_x \mid S_x] = n_x q_{P,x}, \quad \Pr(D_x > 0 \mid S_x) = 1 - (1 - q_{P,x})^{n_x}.$$

The test software also lists distinct problem–outcome pairs. If U_x counts the distinct outside types for problem x , then

$$\mathbb{E}[U_x \mid S_x] = \sum_{o \notin S_x} [1 - (1 - p_P(o \mid x))^{n_x}] \leq n_x q_{P,x}.$$

Each term is the chance of seeing that outside type at least once. Counting distinct types cannot exceed counting every occurrence.

Reference validation concerns $M_{R,x}$, the chance of an unseen answer from the reference. It does not bound $q_{P,x}$, the port’s chance of an outside answer. The port check therefore records outside answers for investigation without treating the reference’s coverage bound as a guarantee about the port.

A budgeted check can also miss rare defects. Suppose the answers caused by a defect have total port probability ϵ and all lie outside S_x . The chance of missing all of them in n_x independent runs is $(1 - \epsilon)^{n_x}$; with 100 runs, detection probabilities for $\epsilon = 10^{-4}, 10^{-3}, 10^{-2}$ are about 0.0100, 0.0952, and 0.6340. Seeing all selected common answers does not make those rare defects easy to detect.

5 How This Relates to Earlier Work

The difficulty of knowing what a test should expect is the *oracle problem* in software testing (Weyuker, 1982; Barr et al., 2015; Patel & Hierons, 2018). Comparing with another program is an established response: it can act as a pseudo-oracle (Davis & Weyuker, 1981), or as the reference in differential testing (McKeeman, 1998). Agreement does not prove both programs correct, and disagreement needs interpretation. Here the variation is intentional, unlike test *flakiness*, where repeated tests unexpectedly change their pass/fail result (Luo et al., 2014).

Another response is *metamorphic testing*: check a relationship between executions when individual expected answers are hard to specify (Chen et al., 1998; 2018; Segura et al., 2016), for example when a known relationship between two inputs implies one between their outputs. Statistical versions and work on stochastic optimization address random outcomes (Guderlei & Mayer, 2007; Yoo, 2010). Such tests could complement ours where domain knowledge supplies reliable relationships. Tests of randomized algorithms also need a stated null hypothesis (the assumption the test is designed to challenge), a sampling protocol, and a specified difference to detect (Arcuri & Briand, 2014). Statistical model checking asks whether particular properties of stochastic executions hold (Younes & Simmons, 2002; Legay et al., 2010). Its guarantees do not come from repeatedly looking at a convenient statistic.

Good–Turing is well established (Good, 1953). Practical frequency estimation (Gale & Sampson, 1995), unseen-vocabulary estimation (Efron & Thisted, 1976), estimating the number of types (Chao, 1984), and coverage-based estimation (Chao & Lee, 1992) address neighbouring, non-interchangeable questions. Bounds on missing mass carry uncertainty terms (McAllester & Schapire, 2000), and the optimality results of Orlitsky et al. (2016) concern predicting numbers of unseen types rather than a stopping rule or port diagnostic.

The closest software-testing precedent is STADS (Böhme, 2018), which treats program behaviours as species to be discovered through sampling. Work on residual risk in greybox fuzzing, which uses execution feedback to generate tests, also estimates the chance of further discoveries (Böhme et al., 2021). Capture–recapture methods have been used to estimate software fault content (Eick et al., 1992). We therefore do not claim to have invented the use of discovery statistics in testing. Our focus is a frozen reference record that repeatedly supplies named-answer comparisons during development. Guarantees for other sampling schemes do not automatically carry over.

Selecting the most frequent outcomes until a target share is reached is an empirical smallest covering region (O’Neill, 2022), which we apply at 50% without claiming a new algorithm or an optimal threshold.

Work on aligning and replicating simulation models distinguishes different levels of agreement (Axtell et al., 1996; Wilensky & Rand, 2007). Our distinction between answer sets and internal processes belongs in that tradition. Differential testing of probabilistic programming systems (Dutta et al., 2018) is a related application we have not tried. Resetting probabilistic stateful systems between fixed-length trials is established (Gerhold & Stoelinga, 2018); that is the relevant precedent for treating an episode as one observation. Missing-mass estimation for Markov sequences (Pananjady et al., 2024) shows that the ordinary singleton estimate is biased once observations are dependent, which is one reason we do not pool singleton counts across a memory history.

Where, then, is the process contribution? It joins construction, freezing, fast checks, and investigation into one reusable workflow for a stochastic port: Good–Turing guides reference collection, p50 heads name common reference solutions missing from a check, and the named differences guide code inspection and repair.

We have not found this complete process in the prior work we examined. That is a bounded comparison, not proof of exclusive priority.

The learning-mode application is part of this contribution, with a boundary: the single-run studies and the historical episodic checks use p50 heads, while the later quality-and-preference study extends construction and independent coverage validation to two episode-answer populations and tests frozen supports only.

6 How the Checks Helped Build the Port

Testing ran alongside the writing of the Python program. An automated check could name a problem and an unexpected answer, which was a concrete starting point for tracing the source, trying an explanation, and checking a repair. Two investigations show how: one found a real direction-handling error, the other a misleading test condition.

6.1 What the Historical Records Establish

The port was developed with language-model assistance, source comparison against the Scheme source, and repeated consultation of the dissertation. Running the program supplied evidence that reading could not, and reading explained differences that running could not.

The benchmark contains 19 letter-string problems from demonstrations and simpler analogy cases. Appendix F lists them all, with examples of answers shared by the reference and port. Individual-run checks clear episodic memory before every run; learning-mode checks retain it within eight-run episodes.

The historical reference was a modified Metacat 1.2 with a 100,000-codelet cap, run *headless*, without the graphical interface. The modifications also addressed unsupported messages sent to program objects, rules acting on whole strings, and message dispatch, and some of them can affect answers. This reference is therefore modified Metacat, and we do not claim the modifications preserve every aspect of its behaviour.

A reconstruction package supplies patches for the linked original release without redistributing its source. Reconstructing the current modified source still does not identify the exact build behind every historical sample; that evidence is missing. The old counts can be audited. The complete historical experiment cannot be reproduced end to end.

The historical single-run checks use seeds 900,000–900,099 for each input. Because the seeds are reused before and after repairs, the two checks are a paired replay.

6.2 The Large Historical Reference

The saved reference contains 374,500 runs and 366 distinct problem–outcome pairs. The same answer to two different problems counts as two pairs. There are 27 selected p50 head members across the 19 problems. Reference sizes range from 10,250 to 51,000 runs, averaging 19,710.5 per problem. Six problems have no singletons; three have singleton ratios above 10^{-4} .

Table 1 gives the full counts. In its headings, $|S_x|$ is the number of different observed outcomes and $|H_x|$ is the number selected for the p50 head. A stored stopping label records what the collector reported.

Table 1: Archived reference counts. Stopping labels describe the heuristic, not certified completeness.

Input	N_x	$ S_x $	$f_{1,x}$	$f_{1,x}/N_x$	Stored reason	$ H_x $
misc4	11,000	4	0	0.00000000	no_singletons	2
fig5.7	18,125	18	1	0.00005517	saturated	2
misc3	19,000	54	4	0.00021053	shards_exited	2
misc5	11,125	8	1	0.00008989	saturated	3
misc2	11,000	6	1	0.00009091	saturated	1
run1	11,000	11	0	0.00000000	no_singletons	1

Input	N_x	$ S_x $	$f_{1,x}$	$f_{1,x}/N_x$	Stored reason	$ H_x $
run4	11,000	10	1	0.00009091	saturated	2
misc1	30,750	18	3	0.00009756	saturated	1
fig5.4-top	51,000	61	19	0.00037255	checkpoint	2
eqe-baaab	51,000	51	13	0.00025490	checkpoint	1
run6	47,500	80	4	0.00008421	saturated	1
run3	21,000	12	2	0.00009524	saturated	2
run2	11,000	10	1	0.00009091	saturated	1
copy1	10,875	2	0	0.00000000	no_singletons	1
copy2	11,250	1	0	0.00000000	no_singletons	1
copy3	11,000	1	0	0.00000000	no_singletons	1
copy4	11,125	2	0	0.00000000	no_singletons	1
copy5	15,500	13	1	0.00006452	saturated	1
copy6	10,250	4	1	0.00009756	saturated	1

saturated means only that the heuristic stopped collection. Two checkpoint cases finished at 51,000 runs because work was already underway. The other above-target case, `misc3`, records `shards_exited`; development notes describe a manual stop, but the aggregate file does not say why the worker processes exited. The spread of collection sizes makes adaptive spending worth considering, though not yet proven better than a fixed budget.

The historical probability calculations are worth keeping, with their assumptions stated. With 100 runs per check, the smallest head-member reference share is $4089/21000 = 0.194714\dots$ for answer `wyz` on problem `run3`. The absence formula gives 3.9355×10^{-10} for it, and the 27 member-specific absence terms add to 6.1340×10^{-10} . Both are plug-in baselines in the sense of Section 4.1.

The saved singleton ratios give 0.169485 expected outside draws per benchmark-wide cycle and 0.167898 expected problems with at least one outside draw. If checks are independent across problems too, the plug-in probability of any outside draw is $1 - \prod_x (1 - \widehat{M}_{R,x})^{100} = 0.155914$. The expected number of distinct outside pairs is bounded above by the expected draw count. A zero singleton ratio contributes zero to these calculations without establishing zero risk.

The rough design figure $19 \times 100 \times 10^{-4} = 0.19$ is therefore no proved upper bound: three recorded estimates exceed the target, and none is an upper confidence limit. The plug-in total happens to fall below 0.19.

6.3 Making the Execution Limits Comparable

One historical check schedules 100 initial runs per problem, or 1,900 across the benchmark, so the reference-to-check ratio is $374500/1900 = 197.105\dots$ in run counts.

The port initially used a working cap of 20,000 codelets against 100,000 in the reference, so a port run could stop before the reference’s budget was exhausted. The repaired test driver reruns capped seeds at 100,000 codelets; the saved post-repair cycle has 23 such reruns, for 1,923 execution attempts. The 197-fold ratio excludes those attempts and any difference in work per attempt.

Rerunning with a larger cap is equivalent to extending the original path only if seeded replay is deterministic and the new cap does not change decisions before the old cutoff. Starting a fresh session is not enough to guarantee that. Equal numerical limits also do not prove equal codelet semantics or equal behaviour across numerical backends. A *backend* is the software-and-hardware route used to execute the calculation. The records include an ordinary CPU (central processing unit) path and a path using the MLX numerical library. They do not establish equivalence between CPU and GPU (graphics processing unit) execution.

6.4 Following the Differences Into the Code

All three saved single-run checks produced every p50 member (Table 2) but disagreed with the reference on some observed outcomes. Before repair, the CPU check reported five outside problem–outcome pairs: `ege-baaab/abbbb`, `run6/cdddb`, `copy1/*NONE*`, `run1/*CAP*`, and `copy5/aac`.

Table 2: Observed flags, not estimated error rates. All cycles reuse the same seed block.

Archived cycle	Novel pairs	Missing head members
CPU before repairs	5	0
CPU after repairs	1	0
Archived MLX check	5	0

The first three led to the direction-image investigation, labelled RC-A in the development record. Here an *image* is an internal representation of letters or groups to which a rule can be applied, not a bitmap picture. A group treated as leftward had its image assembled in physical left-to-right order and marked rightward. Reversing both direction and constituent order can leave the displayed, unchanged string looking correct. That made the error easy to miss until a later operation depended on direction.

The investigation includes a Python execution trace and Scheme-side probes. The development account reports that inserting the suspected defect into the reference produced `cdddb` and `*NONE*`, matching two flagged outcomes. Its intervention table does not show reproduction of `abbbb`. The evidence for that answer is the Python trace and its disappearance after repair, not a claim that every flagged string was recreated in Scheme. Focused regression tests, designed to catch the same error if it returns, provide additional local evidence. These historical instrumented runs were not independently repeated for the paper revision.

The `run1/*CAP*` result, RC-B, had a different explanation: the two programs were allowed different execution budgets, and it disappeared when the caps were reconciled. After the image repair and the driver change, the paired CPU replay contained only `copy5/aac`. One proposed explanation, RC-C, was investigated and rejected; the answer remains unresolved. The separate archived MLX check still contains five outside pairs, so the repairs are unvalidated on that backend.

The useful result is more than five flags reduced to one. The checks located a semantic error, supported a repair and regression tests, distinguished an unfair comparison from a program defect, and kept an unanswered question visible. That is an improvement in specific port behaviour; Section 9 says what it leaves unproved.

Seeing the same flag again helps decide what to investigate first, and no more: a legitimate reference answer omitted from the frozen sample can recur indefinitely, and these historical cycles reuse the same seed block.

6.5 The Earlier Checks With Memory

The historical learning-mode reference is described as 500 eight-run episodes per problem: 9,500 episodes, or 76,000 inner runs. A port check uses 100 episodes per problem: 1,900 episodes and 15,200 inner runs per cycle. The reference-to-check episode ratio is therefore 5:1, far below the 197:1 of the individual-run checks, though both references were meant for reuse.

Those 500 episodes do not establish the single-run heuristic target. Stored episodic singleton ratios reach 0.016, and the available record has no independent missing-mass validation of this reference.

What does memory change? The port retains descriptions of answers and of *snags*, difficulties met while forming an analogy. Between runs it resets the workspace, the concept network, the codelet scheduler, and the current reminding activations, but keeps the stored experiences. The direct effect on later search is the rejection of an answer already remembered under the relevant structural description.

The duplicate check uses the problem, the answer string, the inferred rule, and the translation of that rule to the target, so the same letters reached through a different rule need not be a duplicate. Answer-finding, justification, and recovery from a justification clamp all consult this memory; the clamp is part of the control of justification, and its recovery path must not bypass the check. In the reference source, an answer already present in memory makes the answer-finding codelet fizzle, and the port does the same; search continues, which changes both what later runs find and whether they terminate.

There is also an explanatory role. After storing an answer, the inspected reporting code compares it with earlier answers through rules, themes, justification, and coherence, then records reminding strengths and generates commentary. Here *themes* describe conceptual relationships in an analogy, and *coherence* is the program’s assessment of how well those relationships fit together. Stored snags can change how an otherwise unjustified theme is explained. These are implemented behaviours, not evidence that remembering improves answer quality. The repeated-problem experiment tests exploration conditioned on experience, not learning transferable to new problems.

Existing regression definitions address several memory-related repairs: distinguishing answers with different rules; preventing clamp recovery from accepting a remembered answer; comparing snag rules structurally rather than by potentially identical English descriptions; and assigning zero reminding strength at the relevant distance threshold. Further tests cover distance components, snag-based theme classification, clearing activations without deleting answers, and session-scoped identifiers. The test driver counts a run as successful only if it reports an answer and adds a new memory entry.

These test definitions were inspected for this paper; the engine regressions were not freshly rerun, and Scheme and Python memory paths were not compared. Nor do the historical build records let us attribute each change in the endpoint counts to a particular memory repair.

For the saved port episodes, we recover the last successful answer by looking backward past **NONE** and **CAP**, then check the stored endpoint counts and missing-head lists. Four episodes in each cycle never answer; the old endpoint histogram omits them but records their number, which Table 3 retains.

The historical episodic reference itself is absent from the supplement: its size comes from the development account, and the stored outside-answer lists cannot be regenerated. The saved port sequences do establish how often each listed endpoint occurred.

Table 3: Archived eight-run episodes. Novelty labels come from stored flags; the historical episodic reference is unavailable for independent re-computation.

Metric	CPU before	CPU after	Archived MLX
Check episodes	1,900	1,900	1,900
Within-episode runs	15,200	15,200	15,200
Stored novel endpoint pairs	17	13	17
Episodes with stored novel endpoint	28	14	28
Missing stored head members	0	0	0
Episodes never answering	4	4	4
Capped within-episode runs	1,311	2,404	1,309
Episodes with any capped run	628	1,052	626
Episodes ending in a capped run	361	591	361

The paired CPU records contain 17 outside problem/endpoint pairs across 28 episodes before repair, and 13 pairs across 14 episodes afterward. Neither cycle misses a selected head member. Of the 13 post-repair pairs, 11 also appear in the separate single-run reference. The other two are *misc2/ajd* and *eqe-baaab/qrrbq*, each seen once. That overlap is context only: an answer possible without memory need not have the same probability, or be reachable at all, under a particular learned context.

The development investigation links the direction-image repair to the disappearance of `abbbb`, `baaba`, `cdddb`, and `cddbc` from the episodic outside list, which complements the individual-run findings under the same limits on historical build evidence. The archived MLX sequences are another unvalidated cycle.

Two problems keep these records from being a controlled learning-mode validation. First, the port cap is 20,000 codelets and the reported reference cap is 100,000. An early cap can change memory and therefore every later run in the episode, and similar aggregate cap rates would not make that harmless: capped CPU runs rise from 1,311 to 2,404 after repair even as outside endpoints fall. Later prose reports 2,406 capped runs in 1,054 episodes; the identifiable saved sequences give 2,404 in 1,052, and we use the latter.

Second, episode e , run i uses seed $900000 + 8e + i$, with $0 \leq e < 100$ and $0 \leq i < 8$, so these paired samples overlap the single-run seed block and cannot confirm it independently. Those limitations led to the later study with matching caps and native winners.

7 A Recorded Study of Individual Runs

The later `support-v1a` study records program versions, ordered observations, frozen construction samples, and fresh validation checks, all for one port revision.

7.1 The Protocol and Its Change After a Failure

Every run starts with empty memory. Both programs receive a direct 100,000-codelet limit. For each of the 19 problems, we collect 20,000 reference runs to build the oracle, 30,000 separate reference runs for validation, and 1,000 port runs. The totals are 380,000, 570,000, and 19,000, respectively: 969,000 main observations.

Validation and port observations are divided into prespecified blocks of 100 runs, giving 300 reference checks and ten port checks per problem. The three phases use disjoint seed blocks. A seed equals the recorded phase base plus $100000j + i$, where j indexes the problem and i the run. Independence under a fixed distribution remains a modelling assumption about this pseudorandom sampling.

Reference execution uses Chez Scheme 9.5.4 in a Linux/amd64 environment under emulation. The serial port uses Python 3.14.6 and NumPy 2.5.1. Manifests, inventories of file hashes, record the source, parameter files, dependencies, protocol, and completion receipts.

The original `support-v1` collection stopped when the reference failed reproducibly during validation. Rather than drop the failure or repair the engine mid-collection, we made a separately versioned continuation with the original budgets, problems, seeds, engines, caps, and frozen reference sets. A recognized failure inside the engine becomes `*ERROR*` and the process restarts at the next assigned seed; infrastructure and unclassified failures still stop collection.

Verified complete chunks from the parent study are inherited once. The interrupted chunk is replayed with its original seeds, checking that its 238 successful prefix observations agree. Exactly one observation is admitted for each assigned seed. Incomplete attempts are retained as records of what happened, not extra independent data.

The 969,000 main observations comprise the scheduled construction, validation, and port checks. No engine repair was introduced. To be clear: the continuation is an explicit response to a failure, not an unchanged experiment planned in every detail before it began.

7.2 What the Frozen References Found

The full 20,000-run construction sample for each problem is frozen, along with its p50 head. There are 27 head members across the benchmark. All 3,876 main chunks are complete and pass checksum verification. Table 4 summarizes checks against those references; Appendix C gives the per-problem counts.

Table 4: Versioned fresh-memory checks against 20,000 frozen construction runs per input. Counts pool 19 inputs for accounting, not to estimate a common outcome law. Errors are included in outside draws and also reported separately.

Measure	Reference validation	Port
Observations	570,000	19,000
100-run checks	5,700	190
Outside draws (including errors)	124	8
Checks with outside draws	119	6
Checks missing a p50 member	0	0
Engine-error observations	3	0

New reference data show what the construction missed. Sixteen problems have held-out discoveries: 124 validation draws fall outside construction, including the three program errors. Those draws occur in 119 of 5,700 reference-validation checks.

The other three problems, `misc4`, `misc5`, and `copy6`, have no outside outcomes in 30,000 validation runs each. For one problem on its own, that gives a one-sided 95% upper bound of about 9.9853×10^{-5} . None meets 10^{-4} with simultaneous confidence across all 19 problems: even a zero-miss problem then has an upper bound of about 1.9799×10^{-4} . These are per-problem bounds on the reference.

The port has eight outside draws in six of 190 checks, representing five distinct problem–outcome pairs (Table 5). Six of the eight have counterparts in the new reference observations: direct evidence that a finite construction sample can omit answers the reference produces. We keep the original flags, because validation discoveries never enlarge the frozen oracle.

The remaining two, `misc1/*NONE*` and `copy5/acc`, warrant investigation. No check misses a p50 member.

Table 5: All five flagged port input–outcome pairs. Held-out matches do not enlarge the frozen reference or remove recorded flags.

Input	Outcome	Port draws	Held-out reference draws
<code>misc1</code>	<code>*NONE*</code>	1	0
<code>fig5.4-top</code>	<code>qqiqq</code>	1	5
<code>run6</code>	<code>pqqbc</code>	1	2
<code>copy5</code>	<code>acc</code>	1	0
<code>copy5</code>	<code>cbb</code>	4	2

7.3 The Reference Also Failed

The expensive reference validation exposed three failed executions on allowed inputs. Their seeds and failure signatures are retained:

Input	Seed	Codelets	Exception signature
<code>misc1</code>	20713988	2,835	Non-procedure <code>#f</code>
<code>misc1</code>	20716342	1,981	Non-procedure <code>#f</code>
<code>misc3</code>	20226148	2,088	<code>caddr: incorrect list structure #f</code>

Three executions, two exception signatures, and so far one diagnosis: the first failing continuation identifies a missing letter-category descriptor in `make-group`, and we do not yet know whether the other failures share that cause. Errors stay in the execution denominators and receive separate hard-error flags. The port has no engine errors in this study.

This is another useful development result: building and validating a large oracle stress-tests the original program, and the retained failures are concrete starting points for repair and regression tests. Any sample this large might have found them; we left them in the study.

7.4 Alternative Collection Rules and Their Costs

Because this study saves discovery order, we can ask what would have happened had construction stopped earlier. A *prefix* is the first part of the ordered sample. We compare fixed prefixes of 1,000, 5,000, 10,000, and 20,000 runs per problem, plus two stopping heuristics checked every 500 runs after at least 3,000 observations.

The singleton rule accepts $f_1/N \leq 10^{-4}$, requiring at least 10,000 runs when $f_1 = 0$. The no-discovery rule stops after a gap of at least 1,000 runs without a new outcome. If a rule never fires, it retains the full 20,000-run prefix.

The singleton rule fires for 14 problems. Including the five that reach the limit, it uses 268,500 construction observations and leaves 135 held-out outside draws and nine port outside draws. The no-discovery rule fires for all 19, uses 81,500 observations, and leaves 346 and 34 outside draws, respectively. Table 11 reports all six choices. They share observations and spend different amounts, so the comparison is descriptive; and since the full construction was collected anyway, the prefix totals are hypothetical allocations.

Construction and validation together cost 950,000 reference executions. The ten sets of port checks use 19,000 runs, a 50:1 aggregate count ratio; one benchmark-wide check uses 1,900, so the upfront reference is 500 times one check. The ten checks sample one frozen port. They demonstrate the allocation; savings across a development lifecycle would need successive revisions.

Recorded aggregate attempt wall time is about 466,573 seconds. It adds the durations of inherited, interrupted, continuation, and corrected preflight attempts. Because attempts may overlap, this is neither the campaign’s elapsed duration nor CPU time. Pilot work, earlier diagnosis, wrapper-development preflight, setup, human work, and analysis are not fully included. A complete cost claim would need those too.

8 A Recorded Study of Episodes With Memory

The episodic v3 study covers all 19 problems. Both programs receive eight runs per episode and a 100,000-codelet cap per run, with memory reset between episodes. Each complete episode with an answer contributes a quality winner, `best_a`, and a preference winner, `best_b` (Section 2.4). We compare these answer strings against frozen supports; there is no p50 head for these definitions.

The collection rule. Each answer population freezes at its first eligible $f_1/N \leq 10^{-4}$ crossing, independently of when its partner freezes. Construction ends when both have frozen or after 2,000 episodes for that problem. Four problems reuse the complete 1,000-episode prefixes already collected in an exploratory pilot. Their first eligible freeze is evaluated at that full prefix, not at a reconstructed earlier crossing. The other 15 problems can freeze immediately. This pilot reuse is an explicit part of the design, not an unchanged plan made before the pilot. For new episodes, first seeds are sampled independently with replacement; validation randomness is separate from construction.

If both supports freeze, the problem receives exactly 1,000 reference validation episodes; if construction reaches 2,000 episodes without both crossings, it receives none. Every problem receives 100 port episodes regardless, with descriptive comparison where coverage cannot be claimed. No construction was extended, no validation batch was added, and no frozen set was enlarged after seeing the results.

In total, the study records 10,074 construction, 14,000 validation, and 1,900 port episodes: 25,974 episodes and 207,791 inner runs, including the reused pilot. The one construction error comes from that pilot. Preflight and serializer-diagnostic executions (a serializer writes program results into the saved data format) are overhead outside these totals.

Did the frozen references pass validation? Table 7 reports every problem, including failed and skipped validation. A and B mean quality and preference winners. “Build *E*” counts construction episodes; “Freeze” gives the episode at which the corresponding set stopped changing. A dash means no freeze or no collected

validation; zero misses is written 0. Five `copy5` validation episodes never answer, leaving 995 answered episodes for each of its populations; every other tested population has 1,000.

Table 7: Episodic frozen-oracle coverage. E counts construction episodes; N_v counts answered validation episodes. A and B denote the native quality and conceptual-preference populations. Pass lists definitions meeting the 0.01 coverage target with 95% simultaneous confidence across all 38 populations. Dashes denote an unmet freeze or uncollected validation, not zero misses.

Input	Build E	Freeze A	Freeze B	N_v	Miss A	Miss B	Pass
<code>misc4</code>	2,000	–	1,000	–	–	–	not tested
<code>fig5.7</code>	8	8	6	1,000	37	111	none
<code>misc3</code>	11	11	7	1,000	0	99	A
<code>misc5</code>	2	2	2	1,000	480	591	none
<code>misc2</code>	2	2	2	1,000	6	63	none
<code>run1</code>	2,000	1,000	–	–	–	–	not tested
<code>run4</code>	2,000	–	1,000	–	–	–	not tested
<code>misc1</code>	14	5	14	1,000	589	193	none
<code>fig5.4-top</code>	2,000	–	–	–	–	–	not tested
<code>eqe-baaab</code>	2,000	2	–	–	–	–	not tested
<code>run6</code>	21	21	2	1,000	101	425	none
<code>run3</code>	2	2	2	1,000	377	418	none
<code>run2</code>	2	2	2	1,000	251	242	none
<code>copy1</code>	2	2	2	1,000	0	0	A,B
<code>copy2</code>	2	2	2	1,000	0	0	A,B
<code>copy3</code>	2	2	2	1,000	0	0	A,B
<code>copy4</code>	2	2	2	1,000	2	0	B
<code>copy5</code>	2	2	2	995	36	29	none
<code>copy6</code>	2	2	2	1,000	3	2	none

Only `copy1`, `copy2`, and `copy3` meet the 1% coverage target for both answer populations. Eight of the 28 tested populations pass: those six, `misc3 best_a`, and `copy4 best_b`. The other 20 fail. The ten populations belonging to five construction-limited problems receive no validation. Each passing population has zero misses among 1,000 answered episodes, giving an adjusted upper bound of about 0.006611 under the fixed 38-population allocation.

The early-stopping problem is especially clear in `misc5`. Both sets freeze after just two episodes. Yet validation then produces 480 out of 1,000 quality winners and 591 out of 1,000 preference winners outside those sets. The estimate was zero early, but the reference was far from adequately covered. Independent validation exposed that failure. Fewer construction runs here are not evidence that learning made oracle collection more efficient.

Could we have chosen a better population? Possibly. We chose quality and preference to make the episode’s selected result internally justifiable by Metacat, rather than simply using the last answer. We did not know that the resulting discovery frequencies would suit Good–Turing-guided collection. It may work more effectively with a different set of episodic results, and a future experiment could choose another conceptually meaningful projection to build a better oracle.

That needs a new construction sample and its own validation. The present data cannot separate the effect of the population choice from the effect of stopping immediately, and no change of definition would turn a zero singleton ratio into a confidence certificate. The failed validations stand as results to explain.

What did the port do? Table 8 gives both populations for every problem, validated or not, each with 100 assigned port episodes. Four `copy5` episodes never answer, so its winner denominator is 96; every other denominator is 100. “Outside” counts occurrences outside that population’s frozen set. Where one population froze and its partner did not, membership is reported descriptively for the frozen one.

Among the eight qualified populations, only `misc3 best_a` has outside port winners: 24 in 100 episodes. The other seven have none.

Table 8: Port results for all 19 inputs, each assigned 100 eight-run episodes. N_p counts answered complete episodes; No answer counts entirely answerless episodes; Cap runs counts inner runs reaching the codelet limit (out of 800). Outside counts refer to independently frozen supports. A dash means no frozen support, not zero outside winners. Ref. pass repeats the independently validated reference populations; it is not a port pass/fail verdict.

Input	N_p	No answer	Cap runs	Outside A	Outside B	Ref. pass
misc4	100	0	0	–	0	not tested
fig5.7	100	0	0	2	13	none
misc3	100	0	66	24	26	A
misc5	100	0	0	47	62	none
misc2	100	0	406	0	8	none
run1	100	0	122	0	–	not tested
run4	100	0	0	–	0	not tested
misc1	100	0	97	24	36	none
fig5.4-top	100	0	0	–	–	not tested
ege-baaab	100	0	0	6	–	not tested
run6	100	0	0	7	33	none
run3	100	0	0	45	54	none
run2	100	0	189	23	28	none
copy1	100	0	233	0	0	A,B
copy2	100	0	215	0	0	A,B
copy3	100	0	224	0	0	A,B
copy4	100	0	229	0	0	B
copy5	96	4	0	2	3	none
copy6	100	0	216	0	0	none

Large outside counts also appear against references that failed validation: for `misc5`, 47 quality and 62 preference winners outside in 100 episodes; for `run3`, 45 and 54. With no coverage guarantee behind those sets, the counts are observations only.

Answers and completion are different questions. The port attempts all 15,200 scheduled inner runs. Of these, 9,815 answer, 3,388 stop without an answer below the cap, and 1,997 reach the cap. None has an engine error. Caps occur in 863 episodes, but another run in the same episode may still provide a winner. The four entirely answerless episodes supply neither A nor B and remain in the execution totals. Appendix D accounts for every phase and the inherited error episode. Comparing winners does not test whether the two systems answer or terminate equally often.

8.1 Looking Closely at One Discrepancy

The problem `misc3` is `abc -> aabbcc; kkjjii -> ?`. The example doubles single letters; the target already contains pairs, in descending alphabetical order. Both frozen reference sets contain `kji`, `kkjjii`, and `kkkjjjiii`: one, two, and three copies of each target letter, three different interpretations.

The quality set freezes after 11 episodes with counts 3, 6, and 2 for those strings. The preference set freezes after seven with counts 2, 3, and 2. The observed sets happen to match, but their counts and freeze decisions are separate.

In 1,000 held-out reference episodes, the quality winner is `kji` 466 times, `kkjjii` 319 times, and `kkkjjjiii` 215 times. No quality winner falls outside, so its adjusted upper missing-mass bound is 0.006611, under the 0.01 target. 99 preference winners fall outside; that bound is 0.130501, and that reference fails.

The port’s 100 episodes have 24 outside quality winners across ten strings and 26 outside preference winners across 16. Every outside port string also occurs among the reference validation’s selected *preference* winners, so Metacat can produce all of them; the discrepancy is how often each becomes the winner under each

definition. The failed preference validation limits that comparison to description and leaves the separately qualified quality reference intact.

Could ties explain the quality result? We divide the 24 episodes into three groups: 21 have a single maximum-quality string, one ties among outside strings, and two tie between an outside and an inside string. Only the last two could change membership under a different tie rule, and we retain the specified earliest-occurrence rule. Nine of the outside episodes answer on all eight runs, so truncation cannot explain the whole result either.

Reading the source finds two differences in rule generalization, the replacement of specific changes by a more general rule: the port excludes some literal common changes the reference retains, and it omits some eligibility checks when generalizing changes to an object’s components. Appendix B gives the findings and the saved evidence. The recorded winner descriptions lack the rule and workspace detail needed to assign episodes to either cause. Neither engine has been repaired, and no observation has been removed or reclassified.

This problem illustrates the intended imbalance: 11 construction plus 1,000 validation episodes cost 8,088 reference runs, and the 100-episode port check costs 800. The check identifies a substantial difference worth investigating. Whether memory caused it is open.

9 What Remains Unanswered

The checks helped improve the port, and the later studies expose useful differences. What would be needed to make stronger claims?

Known defects and fair cost comparisons. We have not measured how reliably the workflow detects a controlled collection of defects. That would require deliberately introduced, understood changes, fresh checks, and comparison with alternatives given the same reference-reuse opportunity. The prefix comparisons spend different construction budgets and do not establish superior cost or detection power. Section 4 also shows why these flags are not an unrestricted test of support equality with calibrated error rates.

Recoverable historical evidence. The repair history is central, and its missing build records remain missing. We can audit saved measurements and inspect the documented investigation; not every historical program version or intervention can be recovered. Sections 6.1 and 6.4 state those boundaries, and Appendix A says what the supplement reproduces.

Evidence specific to learning. Memory-retained and memory-cleared episodes, together with known memory defects, would help show which experience-dependent changes these checks detect. The current native-winner study does not diagnose a memory defect, establish faithful memory paths, or separate population choice from immediate stopping. Alternative episode-result definitions remain a future experiment.

Scope beyond this system. The 19 problems and testing parameters were chosen during development, without preregistration, and belong to one cognitive architecture. We have not evaluated continuous or high-dimensional outputs, changing generative pipelines, or transfer to new analogy problems. A different scheduler, numerical precision, or concurrency setting may change the support however harmless the change was meant to be, and matching answer letters can conceal different internal reasoning.

The whole development cost. Neither historical records nor aggregate execution totals include all human investigation and repeated revision costs. A prospective series of repairs with fresh check blocks would measure actual reuse without replaying the same seeds. Comparing development with and without the workflow would help quantify its incremental benefit. The current workflow already automates comparisons that otherwise require manual inspection. What remains unmeasured is the size of that speedup and the total saving after sampling, investigation, and repair are included.

10 Potential Application of the Process

Must this process stop at porting? No. The same investment could support test-driven development of other stochastic systems: define and automate the check up front, then let repeated checks direct attention to the differences.

Consider speeding up a randomized search program without changing the kinds of solutions it finds. An extensively sampled, reviewed version supplies the reference. Before touching the search, define the outcome categories, execution limits, and check budget; let Good–Turing guide collection and independent validation assess coverage. After each change, a budgeted sample reports unexpected solution types and missing common ones, giving the developer specific cases to investigate while unit tests check individual rules.

The same pattern could apply when restructuring a simulator or re-implementing a learning system while preserving selected behaviour. For a system that learns during testing, define the initial state and the unit of observation first; an episode-level result based on the system’s own assessments is one option, and it keeps the check on the behaviour of interest without a fixed answer per run or a comparison of internal sequences.

The reference need not be software at all. Sampled human results could supply an oracle for a system that has yet to be built. Consider a collection of speeches from one consenting speaker, whose characteristic expressions recur across speeches without appearing equally often. First define how to segment the speeches into distinguishable phrasing patterns, so that a new sentence is not automatically a new pattern. A p50 head could then select the patterns accounting for at least half the recorded pattern occurrences. A discovery estimate similar to Good–Turing could guide further collection, with its sampling unit and assumptions adapted to the dependencies within and between speeches. Held-out speeches could help assess the frozen oracle’s coverage. A large language model fine-tuned to imitate the speaker could then be checked, within a fixed generation budget, for missing characteristic patterns and distinguishable patterns outside the observed support set. The check would ask which patterns appear, without prescribing how often each must recur. An outside pattern would prompt inspection, not prove that the person could never use it. This would be a concrete experiment in using human examples to drive implementation and repeated automated testing of a style generator, with pattern definitions reviewed before testing rather than adjusted to accept whatever the model produces.

One distinction from full TDD matters. A sampled reference describes observed behaviour and says nothing about capabilities absent from it. For new behaviour, developers still need a requirement, a trusted example, or a relationship between executions; reference-based checks then guard the behaviour that should stay unchanged while those tests drive the addition. An intentional change in outcomes calls for a reviewed new reference.

The aim is the familiar test-driven loop: define what to check, change the code, inspect the reported failures or differences, and repeat. Here a discrepancy may reflect sampling or an incomplete reference rather than a defect. A check without flags is therefore limited evidence, not a complete pass on the system’s behaviour. These applications are proposals beyond the Metacat case, not additional experiments. They offer a way to extend automated feedback to systems for which a single expected result is the wrong testing abstraction.

11 Conclusion

How do we test a rewrite of a program that is meant to give different answers? Our answer is to collect a large reference, reuse it for fast checks, and investigate the differences those checks name. Good–Turing guides collection of the observed reference set used to flag unexpected port solutions. The p50 head identifies common reference solutions whose absence should be investigated. The comparison is deliberately two-sided: what appeared unexpectedly, and what failed to appear?

Automating those questions removes repeated manual inspection of result distributions from the development loop. Metacat’s own quality and preference judgments make selected conceptual behaviour testable where one fixed expected answer cannot express what the program is meant to do.

In this case the process helped repair a direction-handling error, exposed an execution-limit mismatch, kept unresolved findings visible, and when done as a proper study, exposed further areas of investigation to improve the port. That is the core result.

The later studies make the limits of this method clearer. Some apparently new port answers also appear in fresh reference data. The expensive reference campaign finds failures in the reference itself. For episodes with memory, immediate Good–Turing stopping often freezes a set too early: only three of 19 problems qualify for both winner definitions. Yet one qualified quality reference highlights a clear difference, with 24 outside port winners in 100 episodes against none in 1,000 reference-validation episodes. We do not yet know its cause. Different, internally justifiable episode populations may support better oracle construction, but that needs another experiment.

The novelty is in bringing Good–Turing guidance, reusable support sets, and p50 checks together for stochastic port development, and in extending them to episodic learning results. The ingredients are established. Their value here is that they gave the development work concrete questions to answer.

References

- Andrea Arcuri and Lionel Briand. A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability*, 24(3):219–250, 2014. doi: 10.1002/stvr.1486. URL <https://doi.org/10.1002/stvr.1486>.
- Robert Axtell, Robert Axelrod, Joshua M. Epstein, and Michael D. Cohen. Aligning simulation models: A case study and results. *Computational and Mathematical Organization Theory*, 1(2):123–141, 1996. doi: 10.1007/BF01299065. URL <https://doi.org/10.1007/BF01299065>.
- Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785. URL <https://doi.org/10.1109/TSE.2014.2372785>.
- Marcel Böhme. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology*, 27(2):7:1–7:52, 2018. doi: 10.1145/3210309. URL <https://doi.org/10.1145/3210309>.
- Marcel Böhme, Danushka Liyanage, and Valentin Wüstholtz. Estimating residual risk in greybox fuzzing. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 230–241, 2021. doi: 10.1145/3468264.3468570. URL <https://doi.org/10.1145/3468264.3468570>.
- Anne Chao. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, 11(4):265–270, 1984.
- Anne Chao and Shen-Ming Lee. Estimating the number of classes via sample coverage. *Journal of the American Statistical Association*, 87(417):210–217, 1992. doi: 10.1080/01621459.1992.10475194. URL <https://doi.org/10.1080/01621459.1992.10475194>.
- Tsong Yueh Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, 1998. URL <https://arxiv.org/abs/2002.12543>. Reissued on arXiv in 2020.
- Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018. doi: 10.1145/3143561. URL <https://doi.org/10.1145/3143561>.
- Martin D. Davis and Elaine J. Weyuker. Pseudo-oracles for non-testable programs. In *Proceedings of the ACM ’81 Conference*, pp. 254–257, 1981. doi: 10.1145/800175.809889. URL <https://doi.org/10.1145/800175.809889>.
- Saikat Dutta, Owolabi Legunsen, Zixin Huang, and Sasa Misailovic. Testing probabilistic programming systems. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 574–586, 2018. doi: 10.1145/3236024.3236057. URL <https://doi.org/10.1145/3236024.3236057>.
- Bradley Efron and Ronald Thisted. Estimating the number of unseen species: How many words did Shakespeare know? *Biometrika*, 63(3):435–447, 1976. doi: 10.1093/biomet/63.3.435. URL <https://doi.org/10.1093/biomet/63.3.435>.
- Stephen G. Eick, Catherine R. Loader, M. David Long, Lawrence G. Votta, and Scott Vander Wiel. Estimating software fault content before coding. In *Proceedings of the 14th International Conference on Software Engineering (ICSE)*, pp. 59–65, 1992. doi: 10.1145/143062.143090. URL <https://doi.org/10.1145/143062.143090>.
- William A. Gale and Geoffrey Sampson. Good–Turing frequency estimation without tears. *Journal of Quantitative Linguistics*, 2(3):217–237, 1995. doi: 10.1080/09296179508590051. URL <https://doi.org/10.1080/09296179508590051>.

- Marcus Gerhold and Mariëlle Stoelinga. Model-based testing of probabilistic systems. *Formal Aspects of Computing*, 30(1):77–106, 2018. doi: 10.1007/s00165-017-0440-4. URL <https://doi.org/10.1007/s00165-017-0440-4>.
- I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953. doi: 10.1093/biomet/40.3-4.237. URL <https://doi.org/10.1093/biomet/40.3-4.237>.
- Ralph Guderlei and Johannes Mayer. Statistical metamorphic testing: Testing programs with random output by means of statistical hypothesis tests and metamorphic testing. In *Seventh International Conference on Quality Software (QSIC)*, pp. 404–409, 2007. doi: 10.1109/QSIC.2007.4385527. URL <https://doi.org/10.1109/QSIC.2007.4385527>.
- Douglas R. Hofstadter and Fluid Analogies Research Group. *Fluid Concepts and Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. Basic Books, 1995. ISBN 9780465051540.
- Axel Legay, Benoît Delahaye, and Saddek Bensalem. Statistical model checking: An overview. In *Runtime Verification (RV)*, volume 6418 of *Lecture Notes in Computer Science*, pp. 122–135. Springer, 2010. doi: 10.1007/978-3-642-16612-9_11. URL https://doi.org/10.1007/978-3-642-16612-9_11.
- Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 643–653, 2014. doi: 10.1145/2635868.2635920. URL <https://doi.org/10.1145/2635868.2635920>.
- James B. Marshall. *Metacat: A Self-Watching Cognitive Architecture for Analogy-Making and High-Level Perception*. PhD thesis, Indiana University, Bloomington, 1999. URL <https://science.slc.edu/jmarshall/metacat/dissertation.pdf>.
- James B. Marshall and Douglas R. Hofstadter. The Metacat project: A self-watching model of analogy-making. *Cognitive Studies: Bulletin of the Japanese Cognitive Science Society*, 4(4):57–71, 1997. doi: 10.11225/jcss.4.4_57. URL https://doi.org/10.11225/jcss.4.4_57.
- David A. McAllester and Robert E. Schapire. On the convergence rate of Good–Turing estimators. In *Proceedings of the 13th Annual Conference on Computational Learning Theory (COLT)*, pp. 1–6, 2000. URL <https://www.learningtheory.org/colt2000/papers/McAllesterSchapire.pdf>.
- William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998. URL https://www.bitsavers.org/pdf/dec/dtj/dtj_v10-01_1998.pdf.
- Melanie Mitchell. *Analogy-Making as Perception: A Computer Model*. MIT Press, 1993. URL <https://mitpress.mit.edu/9780262132893/analogy-making-as-perception/>.
- Ben O’Neill. Smallest covering regions and highest density regions for discrete distributions. *Computational Statistics*, 37(3):1229–1254, 2022. doi: 10.1007/s00180-021-01172-6. URL <https://doi.org/10.1007/s00180-021-01172-6>.
- Alon Orlitsky, Ananda Theertha Suresh, and Yihong Wu. Optimal prediction of the number of unseen species. *Proceedings of the National Academy of Sciences*, 113(47):13283–13288, 2016. doi: 10.1073/pnas.1607774113. URL <https://doi.org/10.1073/pnas.1607774113>.
- Ashwin Pananjady, Vidya Muthukumar, and Andrew Thangaraj. Just wing it: Near-optimal estimation of missing mass in a Markovian sequence. *Journal of Machine Learning Research*, 25(312):1–43, 2024. URL <https://jmlr.org/papers/v25/24-0511.html>.
- Krishna Patel and Robert M. Hierons. A mapping study on testing non-testable systems. *Software Quality Journal*, 26(4):1373–1413, 2018. doi: 10.1007/s11219-017-9392-4. URL <https://doi.org/10.1007/s11219-017-9392-4>.

- Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. doi: 10.1109/TSE.2016.2532875. URL <https://eprints.whiterose.ac.uk/id/eprint/110335/>.
- Elaine J. Weyuker. On testing non-testable programs. *The Computer Journal*, 25(4):465–470, 1982. doi: 10.1093/comjnl/25.4.465. URL <https://doi.org/10.1093/comjnl/25.4.465>.
- Uri Wilensky and William Rand. Making models match: Replicating an agent-based model. *Journal of Artificial Societies and Social Simulation*, 10(4), 2007. URL <https://www.jasss.org/10/4/2.html>. Article 2.
- Shin Yoo. Metamorphic testing of stochastic optimisation. In *Third International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pp. 192–201, 2010. doi: 10.1109/ICSTW.2010.26. URL <https://doi.org/10.1109/ICSTW.2010.26>.
- Håkan L. S. Younes and Reid G. Simmons. Probabilistic verification of discrete event systems using acceptance sampling. In *Computer Aided Verification (CAV)*, volume 2404 of *Lecture Notes in Computer Science*, pp. 223–235. Springer, 2002. doi: 10.1007/3-540-45657-0_17. URL https://doi.org/10.1007/3-540-45657-0_17.

A What the Supplement Can Reproduce

The submission attachment contains a standalone LaTeX source bundle for this manuscript and a separate experimental bundle. The latter contains historical measurements, scientific exports from both later studies, the recorded port source and seed data, analysis code, tests, and Metacat reconstruction patches. Its README links results to evidence and gives dependency instructions. Its manifest records included file hashes, original source hashes, replaced documentation, and exclusions.

The original Metacat source archive and runtime binaries are not redistributed. The reconstruction helper downloads the linked upstream release, applies the supplied patches, and checks the resulting files. This supplies the modified reference without silently replacing its original licence or source distribution.

Checking saved data does not repeat the experiments. From the extracted experimental bundle, `python3 verify.py` checks file and frozen-source hashes, regenerates saved-data audit reports and tables, and runs the academic and reconstruction-helper tests. It also reproduces episodic stopping decisions, validation eligibility, bounds, and outcome counts. These checks use the Python standard library and run neither analogy engine. A Unix locking dependency requires macOS, Linux, or a Linux environment such as Windows Subsystem for Linux (WSL).

The historical audits recover p50 heads, stored flags, probability calculations, endpoint counts, and cap accounting from saved measurements. They do not recover missing discovery order, identify all sampled historical builds, or reconstruct historical episodic novelty from an unavailable reference.

The episodic all-problem audit checks both winner populations and their separately frozen sets against the saved episodes. It retains all 38 construction/port count vectors. The `misc3` audit records selected-answer counts, the 24 outside quality-winner events, native winner descriptions, and the diagnostic groups. It does not rebuild unrecorded rule clauses, workspace states, or causal interventions. The public episodic export contains observations and descriptions of selected and tied winners; the full attempt archive is private. The raw-record inspection described in Appendix B is consequently a documented audit finding that the public export alone cannot reproduce.

Four single-run scientific archives preserve ordered observations, raw attempt evidence, receipts, manifests, and collection history. The main observation ledger identifies each counted result; archived copies of attempts add no observations. The lightweight table audit checks saved summaries without repeating the 969,000 engine executions, the manuscript source bundle supplies this version’s tables, and optional reference reconstruction runs no analogy episodes. Checking the reported results needs neither the graphical interface nor new experiments.

For anonymous review, two repository READMEs have replacement instructions. Repository ignore files, a GUI screenshot, private operational metadata, and Git history are omitted. Scientific records, executable source, technical implementation labels, and hash chains remain unchanged. Third-party attribution, the Metacat licence, and the port’s MIT terms are retained; the MIT copyright holder’s display name is anonymized for review.

The bundle is neither a complete user-interface/database deployment nor a ready-configured rerun of the campaign, and packaging changed nothing in the studies.

B The misc3 Evidence in Detail

Table 9 contains every selected-answer count for `misc3`. Each construction column ends at its own freeze point, not the final common construction horizon. Reference validation does not add answers to either frozen set. Counts are one selected string per answered episode, not all tied winners or all answers across its eight runs.

Table 9: Complete `misc3` selected-answer counts. Freeze A and B use 11 and seven construction episodes, respectively; reference validation uses 1,000 episodes and the port check uses 100. Only `kji`, `kkjjii`, and `kkkjjjiii` belong to the frozen supports.

Answer	Freeze A	Freeze B	Ref. A	Ref. B	Port A	Port B
jii	0	0	0	1	0	0
kji	3	2	466	380	25	28
kjii	0	0	0	1	0	1
kjiii	0	0	0	8	5	2
kjji	0	0	0	3	0	1
kjjiii	0	0	0	1	1	1
kjjji	0	0	0	9	2	1
kjjjiii	0	0	0	7	4	1
kkji	0	0	0	2	0	0
kkjiii	0	0	0	1	0	0
kkjji	0	0	0	3	0	0
kkjjii	6	3	319	293	15	19
kkjjiii	0	0	0	4	0	0
kkjjiiii	0	0	0	2	0	0
kkjjjii	0	0	0	5	0	1
kkjjjii	0	0	0	3	0	0
kkjjjiii	0	0	0	3	0	1
kkjjjiiii	0	0	0	3	0	1
kkkjii	0	0	0	9	2	4
kkkjiii	0	0	0	5	3	1
kkkjji	0	0	0	3	1	0
kkkjjii	0	0	0	2	0	1
kkkjjiii	0	0	0	10	4	7
kkkjjji	0	0	0	8	1	1
kkkjjji	0	0	0	4	0	1
kkkjjiii	2	2	215	228	36	27
kkkkjjjii	0	0	0	2	1	1

What the saved descriptions say. All 24 outside port quality winners are also undefeated in the conceptual comparison. They are marked coherent, have three themes and zero unjustified themes, and score from 84 to 88. Unequal numbers of letters in the answer do not by themselves make it incoherent under the native checks.

In 20 of the 24 episodes, saved descriptions also establish that an answer belonging to the frozen set was available: 18 had lower quality, and two tied for highest quality. The other four lack saved evidence of an inside answer, and since the export does not retain every losing description, one may still have occurred.

The three tie cases explain the 21/1/2 grouping in Section 8.1. Using one-based port episode numbers: episode 43 ties `kjjjiii`, `kjjji`, and `kkkjji` at quality 86, all outside. Episode 74 selects `kjjji`, tied with inside answer `kkkjjiii` at 87. Episode 84 selects `kjjjiii`, tied with outside `kkkjjiii` and inside `kkkjjiii` at 86. Selection uses original memory order, which the sorted exports do not preserve, so arrival order for the three ties cannot be independently recovered. Changing tie order cannot remove the other 22 outside memberships.

How the quality score is calculated. Both inspected code paths use $\text{round}(0.6Q_{\text{rule}} + 0.4(100 - T))$. Here Q_{rule} is the native rule-quality score and T is the program’s temperature, an internal control quantity rather than a physical temperature. Rule quality combines uniformity, abstractness, and succinctness: native assessments of consistency, generality, and economy of the rule. Sharing this outer formula does not establish that both programs supply the same inputs to it.

An illustrative calculation shows why rule details matter. At abstractness 96 and uniformity 100, a one-clause rule has succinctness 100 and rule quality 98; three unit-cost clauses have succinctness 67 and rule quality 84. At temperature 10, the resulting answer scores are 95 and 86. Outside strings are not uniformly over-scored in the port, either: reference preference co-winners with string `kkkji` reach quality 88, whereas its two selected port quality winners score 86. The lack of high-scoring port episode maxima is a lead to investigate.

Two source differences, not two proved causes. First, the port’s common-change filter requires an explicitly recorded, non-null relation. The reference excludes only Identity (no change) and can retain a shared literal destination without a named relation. In plain terms, a rule may recognize a common change without expressing it as a named relation such as successor. A letter-to-group object-category change is relevant to grouping `a -> aa`, `b -> bb`, and `c -> cc` under one description. Excluding such schemas, or rule templates, can change which concise component-level rules are available.

Second, the port generalizes common schemas to subobjects without the reference checks for component coverage, shared enclosing objects, and descriptors on uncovered components. Those conditions limit when a change observed in components is eligible for generalization. Omitting them can admit proposals the reference would not make from the same cluster. A subsequent evaluator still assesses rules, however; the missing checks alone do not prove that an invalid proposal survives.

The inspected source hashes match the frozen study manifest and the reconstructed-reference manifest. A read-only audit of the full archive verified all 1,111 `misc3` completion receipts and their 9,699 named files against the public episode and selection records. The 1,011 reference raw exports agree with the published co-winner descriptions and saved earliest selections. This checks that the records agree with each other.

The archive lacks rule clauses, rule-quality components, workspace groupings, and a complete record of memory interventions, so outside episodes cannot be assigned to generation, scoring, translation, or memory causes. This investigation includes no repair, new episode, or enlarged oracle; checking a repaired port needs a separately versioned comparison. Without a matched memory-disabled control, whether learning caused the difference stays open.

C Individual-Run Tables

Table 10 reports all 19 problems in the later fresh-memory study. The construction sets remain frozen. Program errors remain in the validation and port denominators; an error can count both as an outside draw and as a hard error. Those columns therefore overlap. The three problems with no validation discoveries meet the nominal, individual 10^{-4} target, but none meets it under the 19-problem confidence allocation.

Table 10: Versioned single-run results. States counts outcomes in 20,000 construction runs. Reference validation uses 30,000 runs per input and the port 1,000; errors are included in new draws. Flag counts port 100-run checks with an outside draw or absent p50 member, out of ten checks per input.

Input	States	Ref. new	Ref. err.	Port new	Port err.	Flag
misc4	4	0	0	0	0	0
fig5.7	20	2	0	0	0	0
misc3	58	15	1	0	0	0
misc5	8	0	0	0	0	0
misc2	9	1	0	0	0	0
run1	11	1	0	0	0	0
run4	12	1	0	0	0	0
misc1	17	4	2	1	0	1
fig5.4-top	41	28	0	1	0	1
ege-baaab	41	17	0	0	0	0
run6	68	28	0	1	0	1
run3	13	6	0	0	0	0
run2	12	1	0	0	0	0
copy1	2	3	0	0	0	0
copy2	2	1	0	0	0	0
copy3	2	2	0	0	0	0
copy4	2	3	0	0	0	0
copy5	13	11	0	5	0	3
copy6	5	0	0	0	0	0

Table 11 compares the prespecified collection rules on prefixes of the same ordered construction samples, all 20,000 runs of which were collected; the simulated stops describe possible budgets. Each row uses the same validation and port observations.

The singleton rule fails to trigger on five inputs; their full prefixes and later discoveries remain included. The no-discovery rule triggers on every input but leaves more outside reference and port draws. No later discovery is merged into an earlier reference.

Table 11: Shared validation and port observations under ordered-prefix construction. Fired counts triggering inputs (out of 19); untriggered rules retain 20,000 runs. Construction budgets differ.

Construction rule	Build runs	Ref. new	Port new	Fired
Fixed 1,000/input	19,000	991	49	—
Fixed 5,000/input	95,000	383	33	—
Fixed 10,000/input	190,000	229	16	—
Fixed 20,000/input	380,000	124	8	—
Singleton heuristic	268,500	135	9	14
No-discovery heuristic	81,500	346	34	19

D Episode Execution Counts

Keeping the denominators straight. Construction has 10,073 complete answered episodes and one error episode. It has no complete, entirely answerless episodes. Validation has 13,995 answered and five entirely answerless complete episodes. The port has 1,896 answered and four entirely answerless complete episodes. All nine entirely answerless cases are `copy5`. They count toward execution totals but supply no winner.

Table 12: Episodic inner-run counts, including the inherited pilot. The four disjoint outcome columns sum to Attempted; No answer excludes caps and errors.

Phase	Attempted	Answered	No answer	Capped	Error
Construction	80,591	65,108	13,467	2,015	1
Validation	112,000	73,383	20,132	18,485	0
Port	15,200	9,815	3,388	1,997	0

The inherited `run4` construction failure is zero-based episode 46, with first seed 90100368. It attempts seven runs: four answer, two stop without an answer, and the seventh raises an unrecognized `get-bond-facet` message. The eighth is never attempted. Earlier answers stay in the inner-run accounting, but the incomplete episode supplies neither A nor B. This explains 207,791 actual runs instead of $25,974 \times 8 = 207,792$ scheduled runs. The error was not retried as a replacement scientific observation.

Caps affect 1,489 construction, 7,405 validation, and 863 port episodes. These episode counts differ from the capped-run counts in Table 12, because one episode can contain several capped runs and still have a winner from another run.

E Notation

Table 13: Symbols keep observed counts, unknown probabilities, and estimates separate.

Symbol	Meaning
p_R, p_P	Reference and port outcome probabilities under fixed settings
N_x, n_x	Reference and check sizes for problem x , in runs or episodes as specified
K, C_R	Number of check cycles and one-time reference cost
$C_{P,k}, C_{F,k}$	Cost of check k and of investigating its findings
L, ϕ	Runs per episode and the rule selecting an episode’s recorded result
J	Number of problems, 19 here
$c_x(o), f_{1,x}$	Count of outcome o and number of types seen exactly once
S_x, H_x, T_x	Observed reference support set for problem x , p50 head, and observed port-check set
$M_{R,x}, \widehat{M}_{R,x}$	True reference missing mass and its singleton-ratio estimate
$q_{P,x}$	Port probability of an outcome outside the frozen reference set
D_x, U_x	Outside occurrences and distinct outside outcome types

F The Nineteen Problems

Table 14 gives the complete benchmark. For example, the row’s Initial and Modified strings specify the demonstrated change, and Target is the string to which an analogous change is sought. The shared-answer column lists up to three outputs seen in both the historical reference and post-repair CPU check, ordered by decreasing reference count, with lexicographic ties.

Table 14: Benchmark inputs and observed shared answers; examples are not correctness labels.

Input	Initial	Modified	Target	Shared examples
misc4	a	b	z	b, z, y
fig5.7	aabc	aabd	ijkk	ijll, ijl, ijl
misc3	abc	aabbcc	kkjjii	kkjjii, kkkjjjiii, kji
misc5	abc	abd	glz	gld, hlz, glz
misc2	abc	abd	ijk	ijl, ijd, ijk
run1	abc	abd	mrrjjj	mrrkkk, mrrjjk, mrrjjj
run4	abc	abd	xyz	xyd, wyz, xyz
misc1	abc	cba	mrrjjj	jjjrrm, mrrjjj, jrrmmm
fig5.4-top	eeqee	qeeq	xxixx	qeeq, ixxi, ixxq
eqe-baaab	eqe	qeq	abbba	baaab, qeeeq, qbbbq
run6	eqe	qeq	abbbc	qeeeq, baaaq, qcccb
run3	rst	rsu	xyz	xyu, wyz, xyz
run2	xqc	xqd	mrrjjj	mrrkkk, mrrjjk, mrrjjj
copy1	ab	c	ab	c, ab
copy2	bc	d	bc	d
copy3	xy	z	xy	z
copy4	zy	x	zy	x, zy
copy5	aabb	cc	aabb	cc, cac
copy6	abc	d	abc	d, abc